

Use this cheat sheet for various SQL commands and their syntax, descriptions, and examples.

SQLITE

connect()

Syntax: `sqlite3.connect()`

Description: Create a new database and open a database connection to allow sqlite3 to work with it. Call `sqlite3.connect()` to create a connection to the database INSTRUCTOR.db in the current working directory, implicitly creating it if it does not exist.

Example:

```
import sqlite3con = sqlite3.connect("INSTRUCTOR.db")
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



cursor()

Syntax: `con.cursor()`

Description: To execute SQL statements and fetch results from SQL queries, use a database cursor. Call `con.cursor()` to create the Cursor.

Example:

```
cursor_obj = con.cursor()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



execute()

Syntax: `cursor_obj.execute()`

Description: The *execute* method in Python's SQLite library allows to perform SQL commands, including retrieving data from a table using a query like "Select * from table_name." When you execute this command, the result is obtained as a collection of table data stored in an object, typically in the form of a list of lists.

Example:

```
cursor_obj.execute('''insert into INSTRUCTOR values (1, 'Rav', 'Ahuja', 'TORONTO', 'CA')''')
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



fetchall()

Syntax: *cursor_obj.fetchall()*

Description: The *fetchall()* method in Python retrieves all the rows from the result set of a query and presents them as a list of tuples.

Example:

```
statement = '''SELECT * FROM INSTRUCTOR'''
cursor_obj.execute(statement)
output_all = cursor_obj.fetchall()
for row_all in output_all:
    print(row_all)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



fetchmany()

Syntax: *cursor_obj.fetchmany()*

Description: The *fetchmany()* method retrieves the subsequent group of rows from the result set of a query rather than just a single row. To fetch a few rows from the table, use *fetchmany(numberofrows)* and mention how many rows you want to fetch.

Example:

```
statement = '''SELECT * FROM INSTRUCTOR'''
cursor_obj.execute(statement)
output_many = cursor_obj.fetchmany(2)
for row_many in output_many:
    print(row_many)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



`read_sql_query()`

Syntax: `read_sql_query()`

Description: `read_sql_query()` is a function provided by the Pandas library in Python, and it is not specific to MySQL. It is a generic function used for executing SQL queries on various database systems, including MySQL, and retrieving the results as a Pandas DataFrame.

Example:

```
df = pd.read_sql_query("select * from instructor;", conn)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



shape

Syntax: `dataframe.shape`

Description: It provides a tuple indicating the shape of a DataFrame or Series, represented as (number of rows, number of columns).

Example:

```
df.shape
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



close()

Syntax: `con.close()`

Description: `con.close()` is a method used to close the connection to a MySQL database. When called, it terminates the connection, releasing any associated resources and ensuring the

connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your MySQL database interactions.

Example:

```
con.close()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



CREATE TABLE

Syntax: *CREATE TABLE table_name (column1 datatype constraints, column2 datatype constraints, ...);*

Description: The *CREATE TABLE* statement is used to define and create a new table within a database. It specifies the table's name, the structure of its columns (including data types and constraints), and any additional properties such as indexes. This statement essentially sets up the blueprint for organizing and storing data in a structured format within the database.

Example:

```
CREATE TABLE INTERNATIONAL_STUDENT_TEST_SCORES ( <br>country VARCHAR(50), <br>first_name VAR
CHAR(50), <br>last_name VARCHAR(50), <br>test_score INT);
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



barplot()

Syntax: *seaborn.barplot(x="x-axis_variable", y="y-axis_variable", data=data)*

Description: *seaborn.barplot()* is a function in the Seaborn Python data visualization library used to create a bar plot, also known as a bar chart. It is particularly used to display the relationship between a categorical variable and a numeric variable by showing the average value for each category.

Example:

```
import seabornseaborn.barplot(x='Test_Score',y='Frequency', data=dataframe)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



read_csv()

Syntax: `df = pd.read_csv('file_path.csv')`

Description: `read_csv()` is a function in Python's Pandas library used for reading data from a Comma-Separated Values (CSV) file and loading it into a Pandas DataFrame. It's a common method for working with tabular data stored in CSV format

Example:

```
import pandasdf = pandas.read_csv('https://data.cityofchicago.org/resource/jcxq-k9xf.csv')
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



to_sql()

Syntax: `df.to_sql('table_name', index=False)`

Description: `df.to_sql()` is a method in Pandas, a Python data manipulation library used to write the contents of a DataFrame to a SQL database. It allows to take data from a DataFrame and store it structurally within a SQL database table.

Example:

```
import pandasdf = pandas.read_csv('https://data.cityofchicago.org/resource/jcxq-k9xf.csv')df.to_sql("chicago_socioeconomic_data", con, if_exists='replace', index=False, method="multi")
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



read_sql()

Syntax: `df = pd.read_sql(sql_query, conn)`

Description: `read_sql()` is a function provided by the Pandas library in Python for executing SQL queries and retrieving the results into a DataFrame from an SQL database. It's a convenient way to integrate SQL database interactions into your data analysis workflows.

Example:

```
selectQuery = "select * from INSTRUCTOR"df = pandas.read_sql(selectQuery, conn)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DB2

connect()

Syntax: `conn = ibm_db.connect('DATABASE=dbname;HOST=hostname;PORT=port;UID=username;PWD=password;', ", ")`

Description: `ibm_db.connect()` is a Python function provided by the `ibm_db` library, which is used for establishing a connection to an IBM Db2 or IBM Db2 Warehouse database. It's commonly used in applications that need to interact with IBM Db2 databases from Python.

Example:

```
import ibm_dbconn = ibm_db.connect('DATABASE=mydb;HOST=example.com;PORT=50000;UID=myuser;PWD=mypassword;', ' ', '')
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



server_info()

Syntax: `ibm_db.server_info()`

Description: *ibm_db.server_info(conn)* is a Python function provided by the *ibm_db* library, which is used to retrieve information about the IBM Db2 server to which you are connected.

Example:

```
server = ibm_db.server_info(conn)print ("DBMS_NAME: ", server.DBMS_NAME)print ("DBMS_VER:  ", server.DBMS_VER)print ("DB_NAME:   ", server.DB_NAME)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



close()

Syntax: *con.close()*

Description: *con.close()* is a method used to close the connection to a db2 database. When called, it terminates the connection, releasing any associated resources and ensuring the connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your db2 database interactions.

Example:

```
con.close()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



exec_immediate()

Syntax: *sql_statement = "SQL statement goes here" stmt = ibm_db.exec_immediate(conn, sql_statement)*

Description: *ibm_db.exec_immediate()* is a Python function provided by the *ibm_db* library, which is used to execute an SQL statement immediately without the need to prepare or bind it. It's commonly used for executing SQL statements that don't require input parameters or don't need to be prepared in advance.

Example:

```
# Lets first drop the table INSTRUCTOR in case it exists from a previous attempt.dropQuery =  
"drop table INSTRUCTOR"dropStmt = ibm_db.exec_immediate(conn, dropQuery)
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

